In [1]:
```
import pandas as pd
dataset=pd.read_csv("Placement.csv")
dataset
```

Out[1]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p	specialisation	mba_p	status	salary
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed	270000.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed	200000.0
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed	250000.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed	NaN
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed	425000.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed	400000.0
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed	275000.0
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed	295000.0
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed	204000.0
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed	NaN

215 rows × 15 columns

In [2]:
```
dataset.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   sl_no           215 non-null    int64
 1   gender          215 non-null    str
 2   ssc_p           215 non-null    float64
 3   ssc_b           215 non-null    str
 4   hsc_p           215 non-null    float64
 5   hsc_b           215 non-null    str
 6   hsc_s           215 non-null    str
 7   degree_p        215 non-null    float64
 8   degree_t        215 non-null    str
 9   workex          215 non-null    str
 10  etest_p         215 non-null    float64
 11  specialisation  215 non-null    str
 12  mba_p           215 non-null    float64
 13  status          215 non-null    str
 14  salary          148 non-null    float64
dtypes: float64(6), int64(1), str(8)
memory usage: 35.0 KB
```

In [3]:
```
dataset.isnull().sum()
```

Out[3]:

```
sl_no             0
gender            0
ssc_p             0
ssc_b             0
hsc_p             0
hsc_b             0
hsc_s             0
degree_p          0
degree_t          0
workex            0
etest_p           0
specialisation    0
mba_p             0
status            0
salary           67
dtype: int64
```

In [4]:
```
dataset.columns
```

Out[4]:

```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation', 'mba_p',
       'status', 'salary'],
      dtype='str')
```

In [5]:
```
dataset['workex'].dtypes
```

Out[5]:

```
<StringDtype(na_value=nan)>
```

In [6]:
```
dataset['hsc_p'].dtypes
```

Out[6]:

```
dtype('float64')
```

In [7]:
```
for ColumnName in dataset.columns:
    print(ColumnName)
```

```
sl_no
gender
ssc_p
ssc_b
hsc_p
hsc_b
hsc_s
degree_p
degree_t
workex
etest_p
specialisation
mba_p
status
salary
```

In [8]:
```
qual=[]
quan=[]
for ColumnName in dataset.columns:
    if (dataset[ColumnName].dtype=="str"):
```

```
        print("Object:",ColumnName)
        qual.append(ColumnName)
    else:
        print("Integer or Float:",ColumnName)
        quan.append(ColumnName)
```

```
Integer or Float: sl_no
Object: gender
Integer or Float: ssc_p
Object: ssc_b
Integer or Float: hsc_p
Object: hsc_b
Object: hsc_s
Integer or Float: degree_p
Object: degree_t
Object: workex
Integer or Float: etest_p
Object: specialisation
Integer or Float: mba_p
Object: status
Integer or Float: salary
```

In [9]:
```
qual
```

Out[9]:

```
['gender',
 'ssc_b',
 'hsc_b',
 'hsc_s',
 'degree_t',
 'workex',
 'specialisation',
 'status']
```

In [10]:
```
quan
```

Out[10]:

```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

In [11]:
```
def quanqual(dataset):
    qual=[]
    quan=[]
    for ColumnName in dataset.columns:
        if (dataset[ColumnName].dtype=="str"):
            print("Object:",ColumnName)
            qual.append(ColumnName)
        else:
            print("Integer or Float:",ColumnName)
            quan.append(ColumnName)
    return quan,qual
quan,qual =quanqual(dataset)
```

```
Integer or Float: sl_no
Object: gender
Integer or Float: ssc_p
Object: ssc_b
Integer or Float: hsc_p
Object: hsc_b
Object: hsc_s
Integer or Float: degree_p
Object: degree_t
Object: workex
Integer or Float: etest_p
Object: specialisation
Integer or Float: mba_p
Object: status
Integer or Float: salary
```

In [12]:
```
dataset['ssc_p'].mean()
```

Out[12]:

```
np.float64(67.30339534883721)
```

In [13]:
```
dataset['ssc_p'].median()
```

Out[13]:

```
np.float64(67.0)
```

In [14]:
```
dataset['ssc_p'].mode()[0]
```

Out[14]:

```
np.float64(62.0)
```

In [15]:
```
dir(dataset)
```

Out[15]:

```
['T',
 '_AXIS_LEN',
 '_AXIS_ORDERS',
 '_AXIS_TO_AXIS_NUMBER',
 '_HANDLED_TYPES',
 '__abs__',
 '__add__',
 '__and__',
 '__annotations__',
 '__array__',
 '__array_priority__',
 '__array_ufunc__',
 '__arrow_c_stream__',
 '__bool__',
 '__class__',
 '__contains__',
 '__copy__',
 '__dataframe__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__divmod__',
 '__doc__',
 '__eq__',
 '__finalize__',
 '__firstlineno__',
 '__floordiv__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__iand__',
 '__ifloordiv__',
 '__imod__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__invert__',
 '__ior__',
 '__ipow__',
 '__isub__',
 '__iter__',
 '__itruediv__',
 '__ixor__',
 '__le__',
 '__len__',
 '__lt__',
 '__matmul__',
 '__mod__',
 '__module__',
 '__mul__',
 '__ne__',
 '__neg__',
 '__new__',
```

```
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__static_attributes__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_arith_method_with_reindex',
'_attrs',
'_box_col_values',
'_can_fast_transpose',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_from_mgr',
'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_dir_additions',
'_dir_deletions',
```

```
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_get_values_for_csv',
'_getitem_bool_array',
'_getitem_multilevel',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_iset_item',
'_iset_item_mgr',
'_iset_not_inplace',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_module_source',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
```

```
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource_',
'_repr_fits_horizontal_',
'_repr_fits_vertical_',
'_repr_html_',
'_repr_latex_',
'_reset_cache',
'_sanitize_column',
'_select_dtypes_indices',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'asfreq',
'asof',
'assign',
'astype',
'at',
'at_time',
'attrs',
'axes',
'between_time',
'bfill',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
```

```
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'degree_p',
'degree_t',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'etest_p',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'from_dict',
'from_records',
'ge',
'gender',
'get',
'groupby',
'gt',
'head',
'hist',
'hsc_b',
'hsc_p',
'hsc_s',
'iat',
'idxmax',
'idxmin',
'iloc',
'index',
'infer_objects',
'info',
'insert',
'interpolate',
'isetitem',
'isin',
```

```
'isna',
'isnull',
'items',
'iterrows',
'itertuples',
'join',
'keys',
'kurt',
'kurtosis',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'mask',
'max',
'mba_p',
'mean',
'median',
'melt',
'memory_usage',
'merge',
'min',
'mod',
'mode',
'mul',
'multiply',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'pivot',
'pivot_table',
'plot',
'pop',
'pow',
'prod',
'product',
'quantile',
'query',
'radd',
'rank',
'rdiv',
'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'replace',
'resample',
'reset_index',
'rfloordiv',
'rmod',
'rmul',
'rolling',
'round',
```

```
'rpow',
'rsub',
'rtruediv',
'salary',
'sample',
'select_dtypes',
'sem',
'set_axis',
'set_flags',
'set_index',
'shape',
'shift',
'size',
'skew',
'sl_no',
'sort_index',
'sort_values',
'specialisation',
'squeeze',
'ssc_b',
'ssc_p',
'stack',
'status',
'std',
'style',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_hdf',
'to_html',
'to_iceberg',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',
'to_parquet',
'to_period',
'to_pickle',
'to_records',
'to_sql',
'to_stata',
'to_string',
'to_timestamp',
'to_xarray',
'to_xml',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unstack',
```

```
'update',
'value_counts',
'values',
'var',
'where',
'workex',
'xs']
```

In [16]:
```
dataset.describe()
```

Out[16]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

In [17]:
```
descriptive=pd.DataFrame(index=['mean','median','mode'])
descriptive
```

Out[17]:

mean
median
mode

In [18]:
```
descriptive = pd.DataFrame(index = ["Mean", "Median", "Mode"],columns=quan)
descriptive
```

Out[18]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Median	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Mode	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [20]:
```
descriptive.loc['Median','ssc_p']=0
print(descriptive)
```

```
      sl_no ssc_p hsc_p degree_p etest_p mba_p salary
Mean    NaN   NaN   NaN      NaN     NaN   NaN    NaN
Median  NaN     0   NaN      NaN     NaN   NaN    NaN
Mode    NaN   NaN   NaN      NaN     NaN   NaN    NaN
```

In [23]:
```
descriptive=pd.DataFrame(index = ["Mean", "Median", "Mode"],columns=quan)
for ColumnName in quan:
    descriptive.loc["Mean",ColumnName]=dataset[ColumnName].mean()
    descriptive.loc["Median",ColumnName]=dataset[ColumnName].median()
    descriptive.loc["Mode",ColumnName]=dataset[ColumnName].mode()[0]
descriptive
```

Out[23]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
Median	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
Mode	1	62.0	63.0	65.0	60.0	56.7	300000.0

In [25]:
```
dataset['salary']=dataset['salary'].fillna(dataset['salary'].median())
dataset
```

Out[25]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p	specialisation	mba_p	status	salary
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed	270000.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed	200000.0
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed	250000.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed	265000.0
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed	425000.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed	400000.0
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed	275000.0
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed	295000.0
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed	204000.0
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed	265000.0

215 rows × 15 columns

In [26]:
```
dataset.describe()
```

Out[26]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	281283.720930
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	78232.510303
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	250000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	282500.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

In [33]:
```
descriptive=pd.DataFrame(index = ["Mean", "Median", "Mode", "Min", "25%", "50%", "75%", "Max"],columns=quan)
for ColumnName in quan:
    descriptive.loc["Mean",ColumnName]=dataset[ColumnName].mean()
    descriptive.loc["Median",ColumnName]=dataset[ColumnName].median()
    descriptive.loc["Mode",ColumnName]=dataset[ColumnName].mode()[0]
descriptive
```

Out[33]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186	281283.72093
Median	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
Mode	1	62.0	63.0	65.0	60.0	56.7	265000.0
Min	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Max	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [36]:
```
descriptive=pd.DataFrame(index = ["Mean", "Median", "Mode", "Min", "25%", "50%", "75%", "Max"],columns=quan)
for ColumnName in quan:
    descriptive.loc["Mean",ColumnName]=dataset[ColumnName].mean()
    descriptive.loc["Median",ColumnName]=dataset[ColumnName].median()
    descriptive.loc["Mode",ColumnName]=dataset[ColumnName].mode()[0]
    descriptive.loc["Min",ColumnName]=dataset[ColumnName].min()
    descriptive.loc["25%",ColumnName]=dataset[ColumnName].quantile(0.25)
    descriptive.loc["50%",ColumnName]=dataset[ColumnName].quantile(0.50)
    descriptive.loc["75%",ColumnName]=dataset[ColumnName].quantile(0.75)
    descriptive.loc["Max",ColumnName]=dataset[ColumnName].max()
descriptive
```

Out[36]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186	281283.72093
Median	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
Mode	1	62.0	63.0	65.0	60.0	56.7	265000.0
Min	1	40.89	37.0	50.0	50.0	51.21	200000.0
25%	54.5	60.6	60.9	61.0	60.0	57.945	250000.0
50%	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
75%	161.5	75.7	73.0	72.0	83.5	66.255	282500.0
Max	215	89.4	97.7	91.0	98.0	77.89	940000.0

In [42]:
```
descriptive=pd.DataFrame(index = ["Mean", "Median", "Mode", "Min", "25%", "50%", "75%", "Max","IQR","1.5IQR","lesseroutlier","greateroutlier"],columns=quan)
for ColumnName in quan:
    descriptive.loc["Mean",ColumnName]=dataset[ColumnName].mean()
    descriptive.loc["Median",ColumnName]=dataset[ColumnName].median()
    descriptive.loc["Mode",ColumnName]=dataset[ColumnName].mode()[0]
    descriptive.loc["Min",ColumnName]=dataset[ColumnName].min()
    descriptive.loc["25%",ColumnName]=dataset[ColumnName].quantile(0.25)
    descriptive.loc["50%",ColumnName]=dataset[ColumnName].quantile(0.50)
    descriptive.loc["75%",ColumnName]=dataset[ColumnName].quantile(0.75)
    descriptive.loc["Max",ColumnName]=dataset[ColumnName].max()
    descriptive.loc["IQR",ColumnName]=descriptive.loc["75%",ColumnName]-descriptive.loc["25%",ColumnName]
    descriptive.loc["1.5IQR",ColumnName]=1.5*descriptive.loc["IQR",ColumnName]
    descriptive.loc["greateroutlier",ColumnName]=descriptive.loc["75%",ColumnName]+1.5*descriptive.loc["IQR",ColumnName]
    descriptive.loc['lesseroutlier',ColumnName]=descriptive.loc["25%",ColumnName]-1.5*descriptive.loc["IQR",ColumnName]
descriptive
```

Out[42]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186	281283.72093
Median	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
Mode	1	62.0	63.0	65.0	60.0	56.7	265000.0
Min	1	40.89	37.0	50.0	50.0	51.21	200000.0
25%	54.5	60.6	60.9	61.0	60.0	57.945	250000.0
50%	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
75%	161.5	75.7	73.0	72.0	83.5	66.255	282500.0
Max	215	89.4	97.7	91.0	98.0	77.89	940000.0
IQR	107.0	15.1	12.1	11.0	23.5	8.31	32500.0
1.5IQR	160.5	22.65	18.15	16.5	35.25	12.465	48750.0
lesseroutlier	-106.0	37.95	42.75	44.5	24.75	45.48	201250.0
greateroutlier	322.0	98.35	91.15	88.5	118.75	78.72	331250.0

In [73]:
```
descriptive["hsc_p"]["Min"]
```

Out[73]:

```
np.float64(42.75)
```

In [74]:
```
descriptive["hsc_p"]["lesseroutlier"]
```

Out[74]:

```
np.float64(42.75)
```

In [47]:
```
lesser=[]
greater=[]
for ColumnName in quan:
    if(descriptive[ColumnName]["lesseroutlier"]>descriptive[ColumnName]["Min"]):
        lesser.append(ColumnName)
    if(descriptive[ColumnName]["greateroutlier"]<descriptive[ColumnName]["Max"]):
        greater.append(ColumnName)
```

In [48]:
```
lesser
```

Out[48]:

```
['hsc_p', 'salary']
```

In [49]:
```
greater
```

Out[49]:

```
['hsc_p', 'degree_p', 'salary']
```

In [63]:
```
for lesscolumn in lesser:
   dataset.loc[dataset[lesscolumn]<descriptive.loc["lesseroutlier", lesscolumn],
    lesscolumn]=descriptive.loc["lesseroutlier", lesscolumn]
for greatcolumn in greater:
   dataset.loc[dataset[greatcolumn] > descriptive.loc["greateroutlier", greatcolumn],
    greatcolumn]=descriptive.loc["greateroutlier", greatcolumn]
```

In [65]:
```
descriptive=pd.DataFrame(index = ["Mean", "Median", "Mode", "Min", "25%", "50%", "75%", "Max","IQR","1.5IQR","lesseroutlier","greateroutlier"],columns=quan)
for ColumnName in quan:
    descriptive.loc["Mean",ColumnName]=dataset[ColumnName].mean()
    descriptive.loc["Median",ColumnName]=dataset[ColumnName].median()
    descriptive.loc["Mode",ColumnName]=dataset[ColumnName].mode()[0]
    descriptive.loc["Min",ColumnName]=dataset[ColumnName].min()
    descriptive.loc["25%",ColumnName]=dataset[ColumnName].quantile(0.25)
    descriptive.loc["50%",ColumnName]=dataset[ColumnName].quantile(0.50)
    descriptive.loc["75%",ColumnName]=dataset[ColumnName].quantile(0.75)
    descriptive.loc["Max",ColumnName]=dataset[ColumnName].max()
    descriptive.loc["IQR",ColumnName]=descriptive.loc["75%",ColumnName]-descriptive.loc["25%",ColumnName]
    descriptive.loc["1.5IQR",ColumnName]=1.5*descriptive.loc["IQR",ColumnName]
    descriptive.loc["greateroutlier",ColumnName]=descriptive.loc["75%",ColumnName]+1.5*descriptive.loc["IQR",ColumnName]
    descriptive.loc['lesseroutlier',ColumnName]=descriptive.loc["25%",ColumnName]-1.5*descriptive.loc["IQR",ColumnName]
descriptive
```

Out[65]:

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	108.0	67.303395	66.334744	66.358558	72.100558	62.278186	268289.534884
Median	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
Mode	1	62.0	63.0	65.0	60.0	56.7	265000.0
Min	1	40.89	42.75	50.0	50.0	51.21	201250.0
25%	54.5	60.6	60.9	61.0	60.0	57.945	250000.0
50%	108.0	67.0	65.0	66.0	71.0	62.0	265000.0
75%	161.5	75.7	73.0	72.0	83.5	66.255	282500.0
Max	215	89.4	91.15	88.5	98.0	77.89	331250.0
IQR	107.0	15.1	12.1	11.0	23.5	8.31	32500.0
1.5IQR	160.5	22.65	18.15	16.5	35.25	12.465	48750.0
lesseroutlier	-106.0	37.95	42.75	44.5	24.75	45.48	201250.0
greateroutlier	322.0	98.35	91.15	88.5	118.75	78.72	331250.0

In [69]:
```
lesser1=[]
greater1=[]
for ColumnName in quan:
    if(descriptive[ColumnName]["lesseroutlier"]>descriptive[ColumnName]["Min"]):
        lesser1.append(ColumnName)
    if(descriptive[ColumnName]["greateroutlier"]<descriptive[ColumnName]["Max"]):
        greater1.append(ColumnName)
```

In [70]:
```
lesser1
```

Out[70]:

```
[]
```

In [71]:
```
greater1
```

Out[71]:

```
[]
```

In [ ]: